Python Project: The "Bank" System

Objective: Create a banking system using Python classes that handles different account types and allows money transfers between users.

Requirement 1: The Base Class (`Account`)

Create a parent class `Account` with the following attributes and methods:

- **Attributes:** `name`, `balance`, and `min_balance`.
- **Method `deposit(amount)`:** Increases the balance.
- **Method `withdraw(amount)`:** Decreases the balance **only if** the remaining amount is greater than or equal to `min_balance`. Otherwise, print "Insufficient Funds".

Requirement 2: Inheritance

Create two child classes that inherit from `Account`:

1. `Savings`: Should have a `min_balance` of **0** (cannot go into negative).
2. `Current`: Should have a `min_balance` of **-1000** (allows a small overdraft).

- **Method `transfer(amount, target_account)`:**
  - It should try to withdraw the `amount` from the current account.
  - If successful, it should deposit that `amount` into the `target_account` object.
  - If the withdrawal fails, the transfer should be cancelled.

Requirement 3: Testing the Logic

Write a script to perform the following actions:

1. Create a **Savings** account for **"Alice"** with ₹5,000.
2. Create a **Current** account for **"Bob"** with ₹2,000.
3. **Alice** transfers ₹2,000 to **Bob**.
4. **Bob** tries to transfer ₹6,000 to **Alice** (This should fail or work based on his -1000 limit).
5. Print the final balance of both accounts using a `__str__` method.

```python
class Account:
    def __init__(self, name, balance, min_balance):
        self.name = name
        self.balance = balance
        self.min_balance = min_balance

    def deposit(self, amount):
        self.balance += amount
        print(f"Deposited: ₹{amount} to {self.name}.")

    def withdraw(self, amount):
        if self.balance - amount >= self.min_balance:
            self.balance -= amount
            return True  # Success
        else:
            print(f"Transfer Failed: {self.name} has Insufficient Funds!")
            return False # Failure

    # NEW: Transfer Method
    def transfer(self, amount, target_account):
        print(f"\n--- Initiating Transfer: ₹{amount} from {self.name} to
{target_account.name} ---")
        if self.withdraw(amount):
            target_account.deposit(amount)
            print("Transfer Successful!")
        else:
            print("Transfer Cancelled.")

    def __str__(self):
        return f"{self.name}'s Account | Balance: ₹{self.balance}"

class Savings(Account):
    def __init__(self, name, balance):
        super().__init__(name, balance, min_balance=0)

class Current(Account):
    def __init__(self, name, balance):
        super().__init__(name, balance, min_balance=-1000)

# --- Testing the Transfer ---

prateek = Savings("Prateek", 10000)
ravi = Current("Ravi", 5000)

print(prateek)
print(ravi)

# prateek transfers 3000 to Ravi
prateek.transfer(3000, ravi)

print("\n--- Final Balances ---")
print(prateek)
print(ravi)

# Trying a transfer that exceeds min_balance
ravi.transfer(10000, prateek)
```